

Local model optimization benchmark

qwen2.5:0.5b-instruct

Field	Value
Benchmark date	2026-09-20 07:58:26 UTC
Prompt	Custom prompt (custom)
Optimization scope	All optimizations
Objective	Balanced
Configurations tested	36
Total generations	184
Model size / quantization	494.03M / Q4_K_M
Ollama version	0.34.2
Generated by	Ollama Optimizer 1.0.0
Experiment ID	exp_10f9f111db9c

Recommended configuration

temperature = 0.0

Based on the selected evaluation weights and this benchmark's results, this configuration achieved the highest composite score. It is not a universally optimal setting: a different prompt, a different objective or different hardware can change the ranking.

1. Executive summary

This report benchmarks **qwen2.5:0.5b-instruct** against 36 configurations over 184 generations, using the **Balanced** weighting. The run took 710.2 seconds.

Baseline performance

Quality (0-10)	Median latency	Tokens/s	Consistency (0-10)
8.21	3.16 s	25.3	4.09

Tested optimization categories

context, generation, prompt, runtime, system_prompt

Major findings

- 'temperature = 0.0' achieved the highest composite score (7.20/10) under the Balanced weighting.
- Quality is 4.8% higher than baseline; median latency is 22.7% slower; the quality difference is within run-to-run noise at this sample size.
- The quality difference between the recommended configuration and the baseline is within run-to-run variation at this sample size. Treat it as a weak signal and increase runs per configuration to confirm.

Recommended configuration

temperature = 0.0

```
{
  "num_predict": 1024,
  "temperature": 0.0
}
```

2. Model information

Field	Value
Model	qwen2.5:0.5b-instruct
Family	qwen2
Parameters	494.03M
Quantization	Q4_K_M
Format	gguf
Size on disk	379.4 MB
Digest	a8b0c5157701
Last modified	2026-09-20T13:25:30.691905+05:30
Ollama version	0.34.2
Host platform	Windows-10-10.0.26200-SP0
CPU	AMD64 Family 23 Model 24 Stepping 1, AuthenticAMD (4 physical / 8 logical cores)
System RAM	13.89 GB

Field	Value
GPU	none detected (nvidia-smi unavailable)

Quantization

The common default. Good size/quality balance for most local use.

- Memory: lower-precision weights reduce resident size roughly in proportion to the bit width, which is often what decides whether a model fits in VRAM.
- Speed: smaller weights move less data per token, so heavily quantized models are usually faster when memory bandwidth is the bottleneck - but a model that already fits on the GPU may show little difference.
- Quality: quantization is lossy. The loss is usually small at Q5/Q6/Q8 and grows at Q3 and below, and it hits small models harder than large ones.

3. Methodology

Setting	Value
Runs per configuration	5
Maximum generated tokens	1024
Timeout per generation	45.0 s
Streaming	on
Concurrency	1 request(s) at a time
Seed (determinism test)	42
Evaluator	heuristic
Configurations	36
Phases	baseline: 1, validated: 2, coarse: 28, fine: 5

Search strategy

1. Baseline: the model's own defaults with only an output-token cap, measured first.
2. Warm-up: one unmeasured generation loads the model so the first measured run does not include load time.
3. Coarse search: each generation parameter is varied one at a time (OFAT), so cost grows linearly with the number of parameters instead of multiplicatively.
4. Fine search: the best value of each parameter is combined, and the most influential parameter is probed at neighbouring values.
5. Validation: the top-ranked configurations are re-sampled with additional runs to strengthen their statistics.

Metrics collected

- Wall-clock latency measured by the client
- Time to first token (streaming runs only)
- Ollama total_duration, load_duration, prompt_eval_duration, eval_duration
- Prompt tokens and generated tokens as reported by Ollama
- Tokens per second (generated tokens / eval duration)
- System CPU and RAM during each run (psutil, system-wide)
- GPU utilisation and memory (nvidia-smi, when present)

Evaluation methodology

Deterministic heuristics over the produced text.

Criteria and weights: correctness (weight 28%), instruction_following (weight 22%), completeness (weight 16%), format_compliance (weight 16%), relevance (weight 10%), coherence (weight 8%).

- Correctness is only scored where the prompt has a verifiable answer; for open-ended prompts it is reported as not scorable rather than estimated.
- Instruction following counts explicit, checkable constraints (word limits, line counts, forbidden words, required format).
- Consistency combines pairwise content-word similarity across runs with the stability of the quality score.

- Scores measure observable output only. They do not assess hidden reasoning, and for open-ended prompts they do not verify factual accuracy.

LLM judge: Not used in this experiment.

Test prompt

What is you name and how can you help me?

4. Baseline results

No sampling parameters are sent, so Ollama applies the values baked into the model's Modelfile. This is the reference point for every comparison in this report.

5 of 5 runs succeeded.

Measurement	Value	Uncertainty / note
Quality (0-10)	8.21	95% CI [7.37, 9.05]
Median latency	3.160 s	-
Mean latency	3.512 s	95% CI [2.46, 4.57] s
Latency min / max	2.488 / 4.448 s	-
Time to first token	0.123 s	-
Tokens per second	25.27	95% CI [22.41, 28.14]
Generated tokens	84.4	-
Prompt tokens	40.0	-
Total duration (Ollama)	3.503 s	-
Prompt eval duration	0.093 s	-
Generation duration	3.385 s	-
Load duration	0.014 s	-
Consistency (0-10)	4.09	-
Failure rate	0%	-
CPU during runs	62.5%	-
RAM during runs	66.1%	-
GPU utilisation	N/A - metric unavailable on this system	-
GPU memory used	N/A - metric unavailable on this system	-

Representative baseline output

As an AI language model, I am named Qwen. I was created by Alibaba Cloud based on the knowledge base of artificial intelligence and natural language processing. My purpose is to assist users in generating human-like responses, answering questions, providing information, and performing other tasks that involve language processing and artificial intelligence. If you have any specific questions or need assistance with a particular topic, feel free to ask!

5. Optimization results

5.1 Context optimization

What changed: The context window (num_ctx) and/or the layout of the prompt content were changed.

Why it was tested: Tests context-window sizes the model actually supports, and - for long prompts - whether trimming or reordering the context changes quality, latency or memory behaviour.

Configuration	Phase	Quality	Median latency	Tokens/s	Consistency	Failures
num_ctx = 8192	coarse	8.49	4.494 s	20.2	5.19	0%
num_ctx = 2048	coarse	8.54	3.375 s	20.4	5.06	0%
num_ctx = 32768	coarse	8.66	3.496 s	20.1	5.71	0%

The strongest configuration in this category was 'num_ctx = 32768' (quality 8.66/10, median latency 3.496 s). Quality is 5.5% higher than baseline; median latency is 10.6% slower; the quality difference is within run-to-run noise at this sample size.

num_ctx = 8192 - Sets the context window to 8192 tokens (the model declares 32768; this prompt needs roughly 1164). A smaller window reduces KV-cache memory and can speed up prompt evaluation; a larger one costs memory for no benefit when the prompt does not need it. Changing num_ctx reloads the model, so load time is reported separately.

num_ctx = 2048 - Sets the context window to 2048 tokens (the model declares 32768; this prompt needs roughly 1164). A smaller window reduces KV-cache memory and can speed up prompt evaluation; a larger one costs memory for no benefit when the prompt does not need it. Changing num_ctx reloads the model, so load time is reported separately.

num_ctx = 32768 - Sets the context window to 32768 tokens (the model declares 32768; this prompt needs roughly 1164). A smaller window reduces KV-cache memory and can speed up prompt evaluation; a larger one costs memory for no benefit when the prompt does not need it. Changing num_ctx reloads the model, so load time is reported separately.

5.2 Generation parameters

What changed: Sampling parameters varied: repeat_penalty, seed, temperature, top_k, top_p.

Why it was tested: Sweeps the sampling parameters Ollama exposes (temperature, top_p, top_k, repeat_penalty) one factor at a time, then combines the best value of each.

Configuration	Phase	Quality	Median latency	Tokens/s	Consistency	Failures
repeat_penalty = 1.2	coarse	8.21	4.176 s	19.0	3.86	0%
temperature = 1.15 (fine probe)	fine	8.21	3.430 s	18.3	4.46	0%
top_p = 1.0	coarse	8.26	3.775 s	19.7	4.33	0%
top_p = 0.9	coarse	8.21	2.874 s	20.2	4.40	0%
top_k = 40	coarse	8.49	4.407 s	19.3	4.66	0%
Best value of each parameter, combined	fine	8.49	4.502 s	19.3	4.75	0%
temperature = 0.7	coarse	8.49	4.315 s	19.8	4.92	0%
temperature = 0.85 (fine probe)	fine	8.43	3.260 s	19.1	4.85	0%
top_k = 20	coarse	8.66	4.171 s	19.7	5.63	0%
repeat_penalty = 1.1	coarse	8.77	3.549 s	18.5	5.06	0%
temperature = 1.0	coarse	8.77	3.860 s	19.7	5.32	0%
temperature = 0.3	coarse	8.66	4.645 s	20.5	6.29	0%

Configuration	Phase	Quality	Median latency	Tokens/s	Consistency	Failures
top_k = 100	coarse	8.66	3.403 s	20.1	5.52	0%
repeat_penalty = 1.0	coarse	8.66	3.216 s	20.0	5.73	0%
top_p = 0.7	coarse	8.82	4.357 s	20.3	5.83	0%
seed = 42 (fixed)	validated	8.60	4.150 s	20.1	10.00	0%
temperature = 0.0	validated	8.60	3.878 s	21.2	9.95	0%

The strongest configuration in this category was 'top_p = 0.7' (quality 8.82/10, median latency 4.357 s). Quality is 7.5% higher than baseline; median latency is 37.9% slower; the quality difference is within run-to-run noise at this sample size.

repeat_penalty = 1.2 - Penalises tokens that already appeared. Higher values suppress looping but can distort required repetition such as JSON keys or list markers, so both directions are tested.

temperature = 1.15 (fine probe) - Temperature 1.0 won the coarse sweep, so the neighbouring value 1.15 is tested to check whether the coarse grid landed on a local peak or merely near one.

top_p = 1.0 - Nucleus sampling keeps the smallest set of tokens whose cumulative probability reaches top_p. Tightening it trims the low-probability tail, which can reduce drift but also reduce variety.

5.3 Prompt optimization

What changed: The prompt text was rewritten; the model, sampling settings and system prompt were held constant.

Why it was tested: Rewrites the user's prompt into structured variants (explicit task framing, output-format specification, constraints, step decomposition, condensed form) and measures whether the rewrite changes observable output quality.

Configuration	Phase	Quality	Median latency	Tokens/s	Consistency	Failures
Explicit requirements checklist	coarse	7.79	3.416 s	20.0	6.21	0%
Role and context framing	coarse	7.78	3.246 s	20.3	6.17	0%
Output-format specification only	coarse	7.85	2.344 s	19.8	6.39	0%
Structured task framing + temperature 0.2	fine	7.61	2.173 s	20.0	8.32	0%
Structured task framing	coarse	7.93	1.985 s	20.4	6.02	0%

The strongest configuration in this category was 'Structured task framing' (quality 7.93/10, median latency 1.985 s). Quality is 3.4% lower than baseline; median latency is 37.2% faster; the quality difference is within run-to-run noise at this sample size.

Explicit requirements checklist - Restates the detected constraints as an explicit checklist. Tests whether enumerating requirements improves instruction following.

Role and context framing - Prefixes a short role statement. Tests the common claim that assigning a role improves domain-appropriate output; the claim is measured, not assumed.

Output-format specification only - Adds only the output contract, changing nothing else. Isolates the effect of format instructions from the other rewrites.

Prompt variant used by 'Explicit requirements checklist'

Task:
What is your name and how can you help me?

Requirements:
- answers the prompt (no explicit constraints detected)

Output: a direct prose answer with no preamble.

5.4 Runtime / inference

What changed: Runtime settings were changed: streaming mode, CPU thread count and batch size.

Why it was tested: Tests runtime-level settings: streaming versus a single buffered response, CPU thread count, and batch size. Settings that would require reloading the model onto different hardware, or restarting the Ollama server, are reported as not tested.

Configuration	Phase	Quality	Median latency	Tokens/s	Consistency	Failures
num_batch = 256	coarse	8.66	4.381 s	19.4	5.88	0%
num_batch = 1024	coarse	8.77	4.856 s	20.3	5.49	0%
num_thread = 4	coarse	8.71	3.754 s	20.7	6.07	0%
Non-streaming response	coarse	8.82	3.419 s	20.4	5.96	0%

The strongest configuration in this category was 'Non-streaming response' (quality 8.82/10, median latency 3.419 s). Quality is 7.5% higher than baseline; median latency is 8.2% slower; the quality difference is within run-to-run noise at this sample size.

num_batch = 256 - Sets the prompt-processing batch size to 256 (Ollama's default is 512). Larger batches can speed up prompt evaluation at the cost of memory; smaller batches can help on constrained hardware. The effect is mostly on prompt-eval time, so it matters more for long prompts than for long outputs.

num_batch = 1024 - Sets the prompt-processing batch size to 1024 (Ollama's default is 512). Larger batches can speed up prompt evaluation at the cost of memory; smaller batches can help on constrained hardware. The effect is mostly on prompt-eval time, so it matters more for long prompts than for long outputs.

num_thread = 4 - Pins inference to 4 CPU threads (this machine reports 4 physical / 8 logical cores). More threads are not always faster: oversubscription causes contention, and on a GPU-resident model the setting may have no effect at all. Changing it reloads the model.

5.5 System prompt

What changed: A system message was supplied; the user prompt and sampling settings were held constant.

Why it was tested: Compares system-prompt strategies (minimal, accuracy-focused, concise, structured-output, plus a task-specific one) against sending no system prompt.

Configuration	Phase	Quality	Median latency	Tokens/s	Consistency	Failures
System prompt: Accuracy-focused	coarse	8.66	2.151 s	20.2	5.36	0%
System prompt: task-specific	coarse	8.77	2.616 s	20.6	4.80	0%
System prompt: Minimal system instruction	coarse	8.82	2.953 s	20.4	5.09	0%
System prompt: Structured-output	coarse	8.88	2.285 s	20.1	5.57	0%
System prompt: Concise-response + temperature 0.2	fine	8.66	1.695 s	19.0	7.32	0%
System prompt: Concise-response	coarse	8.88	2.195 s	20.6	5.87	0%

The strongest configuration in this category was 'System prompt: Structured-output' (quality 8.88/10, median latency 2.285 s). Quality is 8.2% higher than baseline; median latency is 27.7% faster; the quality difference is within run-to-run noise at this sample size.

System prompt: Accuracy-focused - Tests whether an accuracy directive changes measurable output quality or simply makes the model more hedging.

System prompt: task-specific - A system prompt written for this specific task type. Tests whether task-specific framing beats the generic variants for this workload.

System prompt: Minimal system instruction - The shortest plausible system prompt. Acts as the control for this category: if richer system prompts do not beat it, they are not worth the tokens.

System prompt used by 'System prompt: Accuracy-focused'

You are a precise assistant. Answer only what was asked. If you are not certain of a fact, say so rather than guessing.

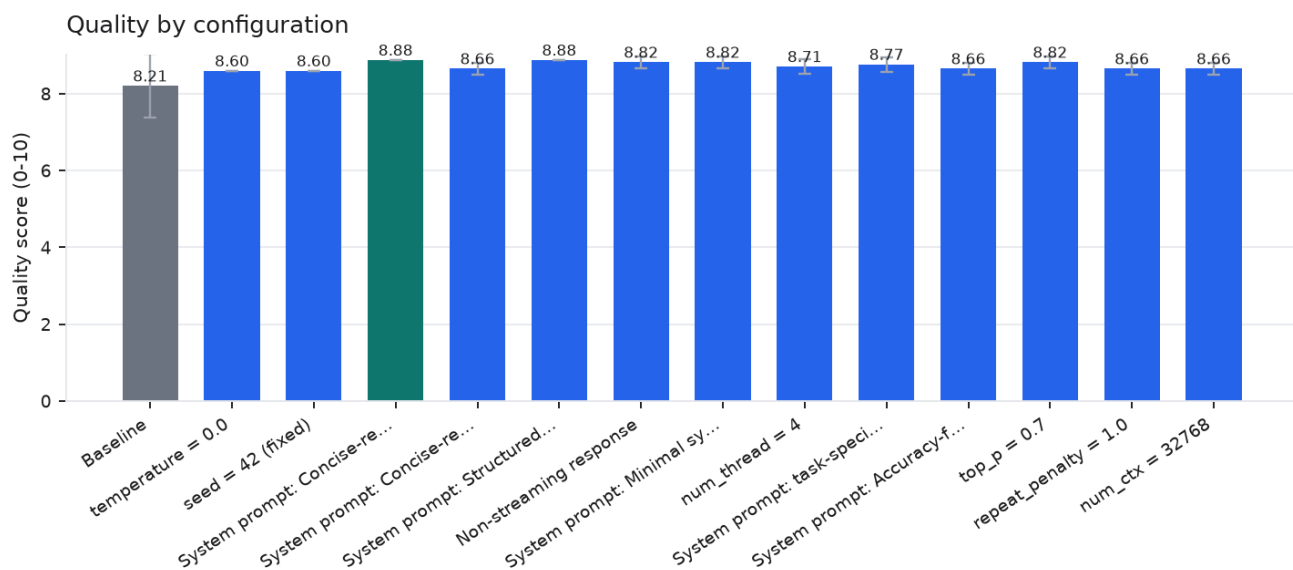
6. Comparative analysis

#	Configuration	Quality	Speed	Consistency	Avg latency	Tokens/s	Composite
1	temperature = 0.0	8.60	3.69	9.95	4.27s	21.2	7.20
2	seed = 42 (fixed)	8.60	2.38	10.00	4.19s	20.1	6.83
3	System prompt: Concise-response	8.88	5.30	5.87	2.19s	20.6	6.57
4	System prompt: Concise-response + temperature 0.2	8.66	4.57	7.32	1.93s	19.0	6.36
5	System prompt: Structured-output	8.88	4.78	5.57	2.29s	20.1	6.27
6	Non-streaming response	8.82	3.60	5.96	3.59s	20.4	5.93
7	System prompt: Minimal system instruction	8.82	4.18	5.09	2.58s	20.4	5.68
8	num_thread = 4	8.71	3.43	6.07	3.87s	20.7	5.58
9	System prompt: task-specific	8.77	4.81	4.80	3.97s	20.6	5.55
10	System prompt: Accuracy-focused	8.66	5.00	5.36	2.07s	20.2	5.53
11	top_p = 0.7	8.82	2.29	5.83	5.88s	20.3	5.47
12	repeat_penalty = 1.0	8.66	3.54	5.73	3.44s	20.0	5.27
13	num_ctx = 32768	8.66	3.25	5.71	3.63s	20.1	5.17
14	top_k = 100	8.66	3.35	5.52	3.64s	20.1	5.11
15	temperature = 0.3	8.66	2.09	6.29	5.88s	20.5	5.11
16	temperature = 1.0	8.77	2.44	5.32	6.02s	19.7	5.09
17	num_batch = 1024	8.77	1.66	5.49	5.16s	20.3	4.94
18	repeat_penalty = 1.1	8.77	1.79	5.06	4.06s	18.5	4.77
19	top_k = 20	8.66	2.03	5.63	5.02s	19.7	4.77
20	num_batch = 256	8.66	1.48	5.88	4.58s	19.4	4.73
21	num_ctx = 2048	8.54	3.66	5.06	4.14s	20.4	4.63
22	Baseline (model defaults) [baseline]	8.21	8.15	4.09	3.51s	25.3	4.44
23	num_ctx = 8192	8.49	2.06	5.19	5.87s	20.2	4.04
24	temperature = 0.85 (fine probe)	8.43	2.71	4.85	3.29s	19.1	3.89
25	temperature = 0.7	8.49	1.94	4.92	4.40s	19.8	3.87
26	Structured task framing	7.93	5.39	6.02	2.24s	20.4	3.68
27	Structured task framing + temperature 0.2	7.61	4.84	8.32	2.18s	20.0	3.63
28	top_k = 40	8.49	1.42	4.66	4.31s	19.3	3.58
29	Best value of each parameter, combined	8.49	1.25	4.75	4.71s	19.3	3.58
30	top_p = 0.9	8.21	4.07	4.40	3.73s	20.2	3.37
31	Output-format specification only	7.85	4.41	6.39	2.65s	19.8	3.33
32	top_p = 1.0	8.26	2.58	4.33	3.54s	19.7	3.07

#	Configuration	Quality	Speed	Consistency	Avg latency	Tokens/s	Composite
33	Role and context framing	7.78	3.74	6.17	2.97s	20.3	2.79
34	temperature = 1.15 (fine probe)	8.21	1.80	4.46	3.39s	18.3	2.72
35	Explicit requirements checklist	7.79	3.23	6.21	3.40s	20.0	2.69
36	repeat_penalty = 1.2	8.21	1.43	3.86	4.63s	19.0	2.31

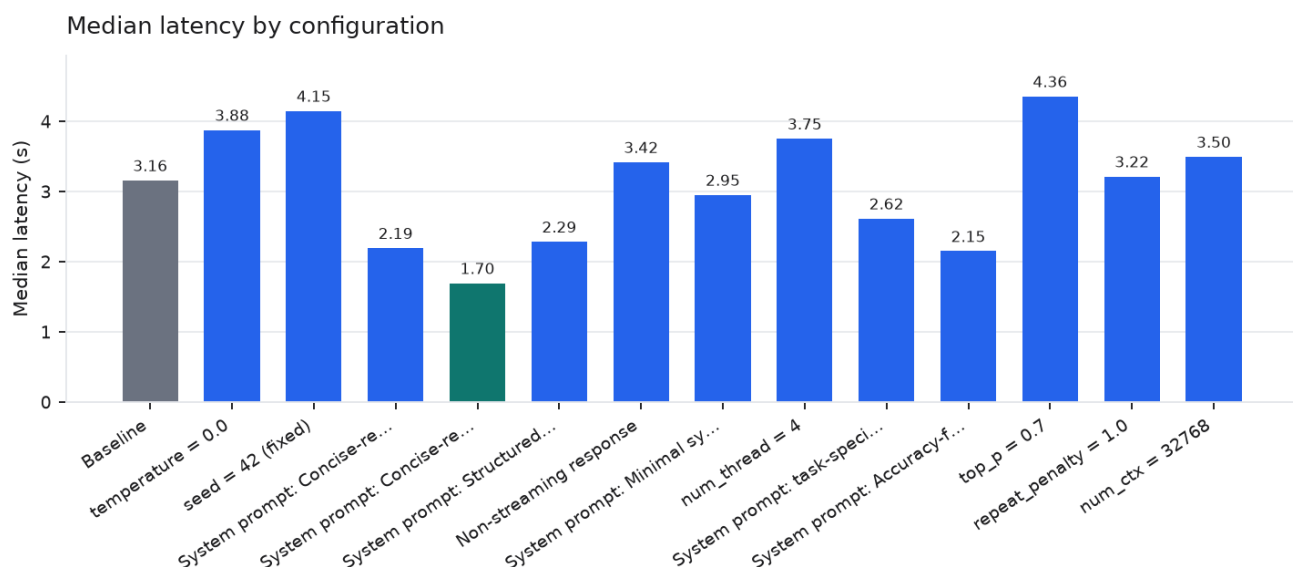
Composite is the weighted combination of the normalised component scores; the raw measurements behind it are in the columns to its left and in the per-run detail.

Quality by configuration



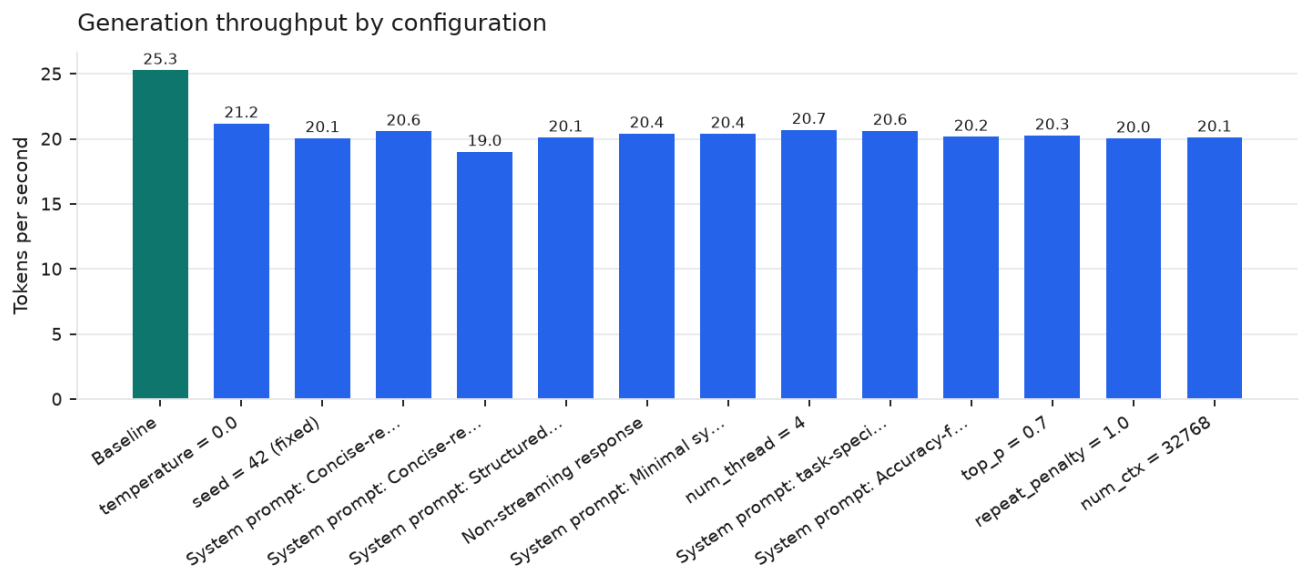
Grey = baseline, blue = tested configuration, teal = best on this metric. Error bars show the 95% confidence interval where one could be computed.

Median latency by configuration



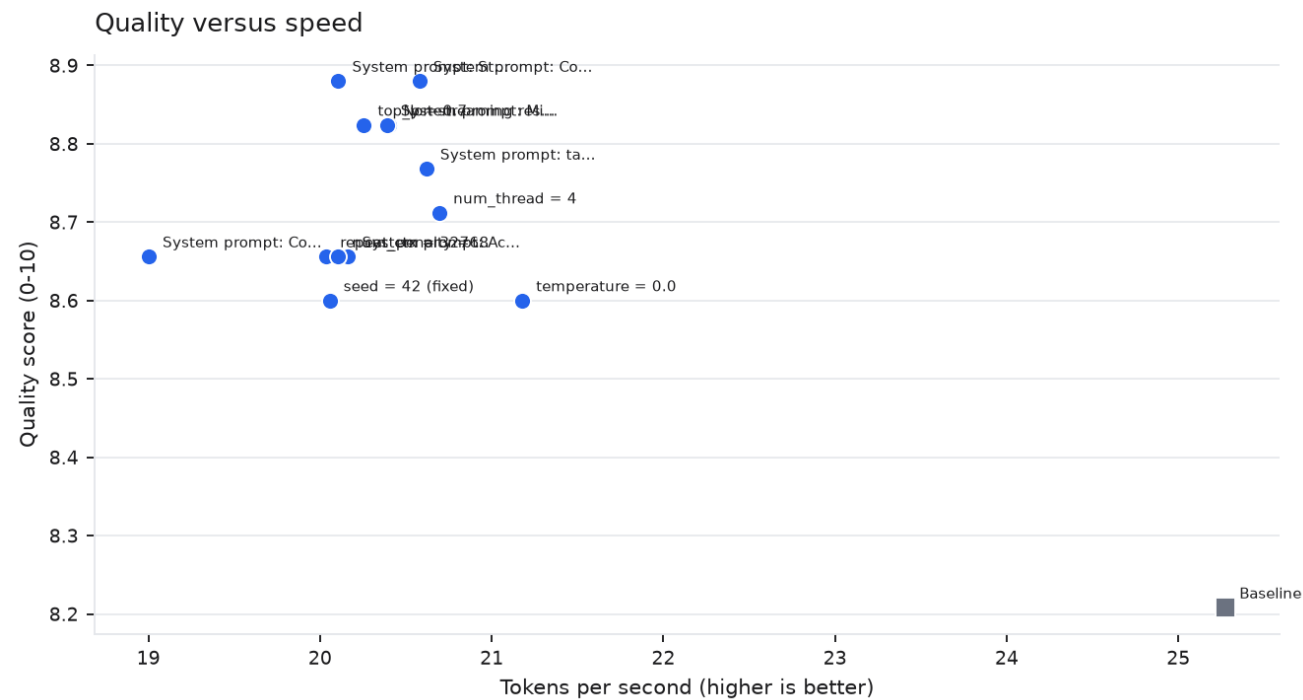
Grey = baseline, blue = tested configuration, teal = best on this metric.

Generation throughput by configuration



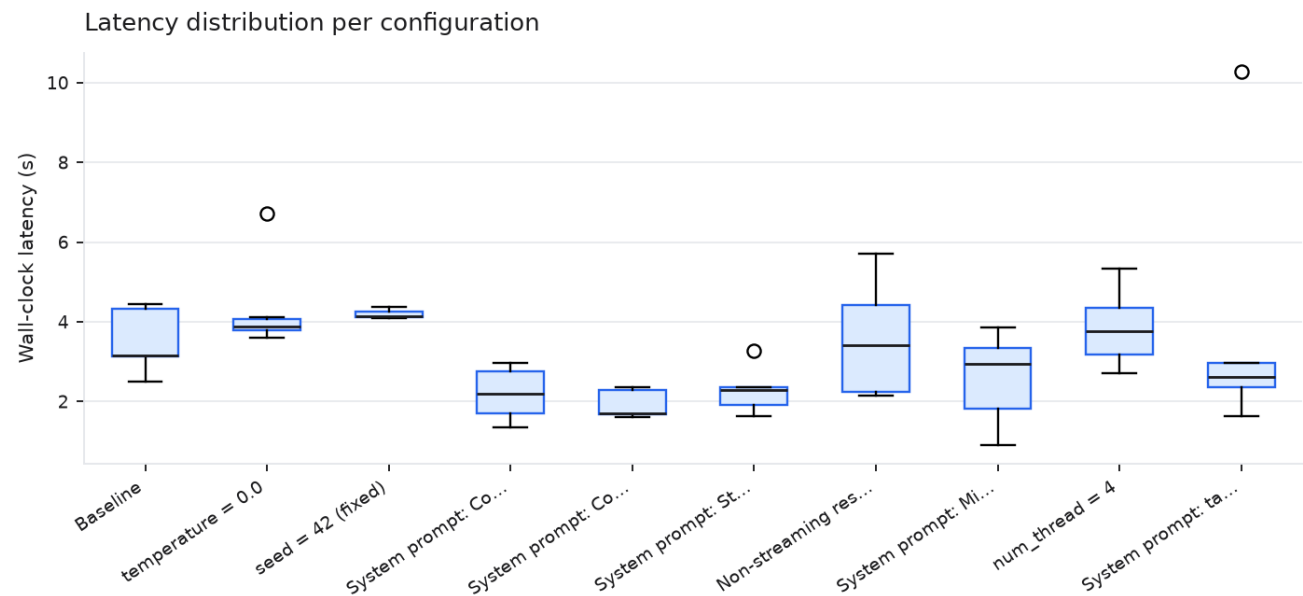
Grey = baseline, blue = tested configuration, teal = best on this metric.

Quality versus speed



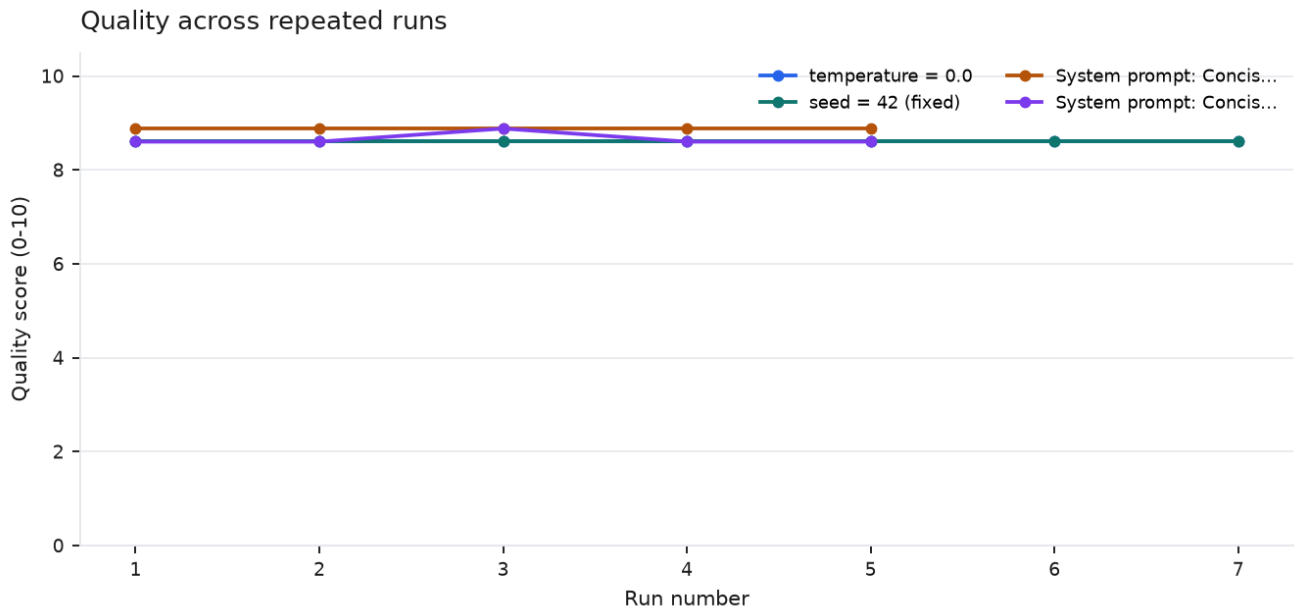
Top-right is the desirable corner: fast and high scoring. The square marker is the baseline.

Latency distribution per configuration



Each box spans the interquartile range of that configuration's runs, with the median marked. Wide boxes mean unstable latency.

Quality across repeated runs



A flat line means the configuration scored the same on every repeat; a jagged line means run-to-run variation that a single test would have hidden.

7. Optimization trade-offs

temperature = 0.0

- + quality +4.8%
- + consistency +143.3%
- tokens/sec -16.2%
- median latency +22.7%

seed = 42 (fixed)

- + quality +4.8%
- + consistency +144.5%
- tokens/sec -20.6%
- median latency +31.3%

System prompt: Concise-response

- + quality +8.2%
- + median latency -30.6%
- + consistency +43.5%
- tokens/sec -18.6%

System prompt: Concise-response + temperature 0.2

- + quality +5.5%
- + median latency -46.4%
- + consistency +79.0%
- tokens/sec -24.8%

System prompt: Structured-output

- + quality +8.2%
- + median latency -27.7%
- + consistency +36.2%
- tokens/sec -20.4%

Non-streaming response

- + quality +7.5%
- + consistency +45.7%
- tokens/sec -19.3%
- median latency +8.2%

8. Recommended configuration

Field	Value
Model	qwen2.5:0.5b-instruct
Objective	Balanced
Weights	Quality (0-10 heuristic score): 40%, Speed (tokens/sec and median latency): 30%, Consistency across repeated runs: 30%
Configuration	temperature = 0.0

Configuration

```
{
  "num_predict": 1024,
  "temperature": 0.0
}
```

Observed results

Quality	Tokens/s	Median latency	Consistency	Runs
8.60/10	21.2	3.88 s	9.95/10	7

95% CI [8.60, 8.60]

Compared with baseline

Quality	Tokens/s	Median latency	Consistency
+4.8%	-16.2%	+22.7%	+143.3%

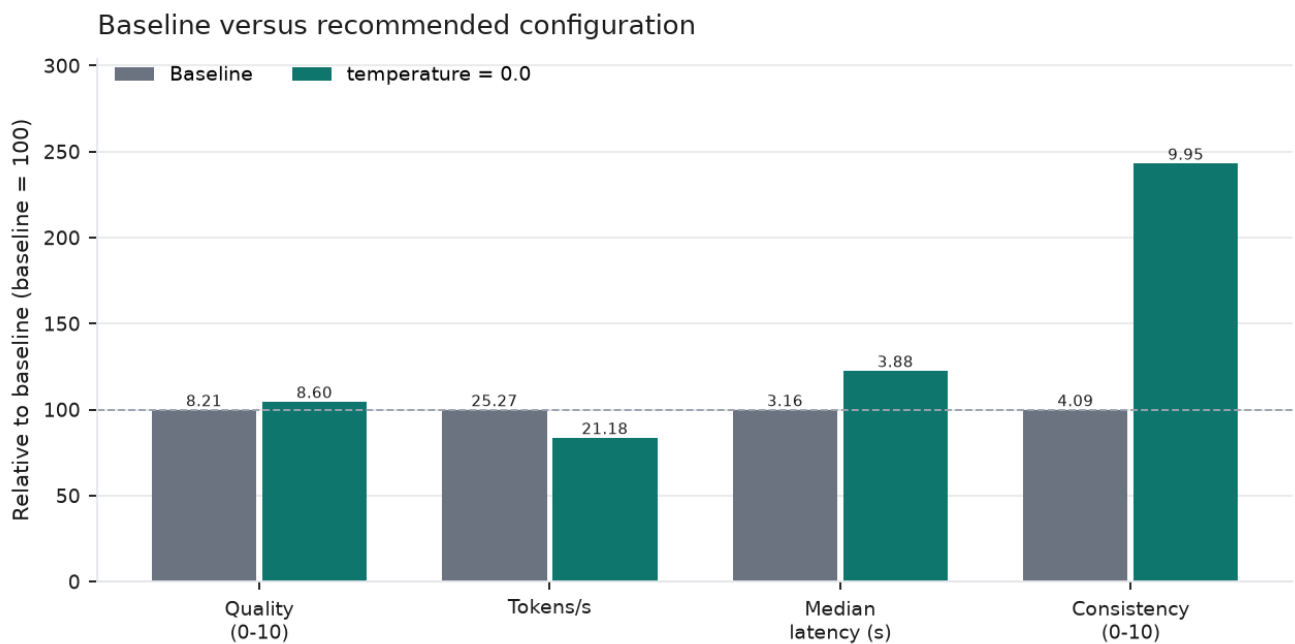
Welch's t-test on a small sample; treat as indicative only

Why this configuration was selected

'temperature = 0.0' ranked first. Quality (0-10 heuristic score) scored 7.80/10 and carries weight 40%, contributing 3.12; Speed (tokens/sec and median latency) scored 3.69/10 and carries weight 30%, contributing 1.11; Consistency across repeated runs scored 9.92/10 and carries weight 30%, contributing 2.98. The weighted contributions sum to 7.20 over 100% of the available weight, giving a composite score of 7.20/10. Against baseline: Quality is 4.8% higher than baseline; median latency is 22.7% slower; the quality difference is within run-to-run noise at this sample size.

Based on the selected evaluation weights and this benchmark's results, this configuration achieved the highest composite score. It is not a universally optimal setting: a different prompt, a different objective or different hardware can change the ranking.

Baseline versus recommended configuration



Bars are scaled so the baseline is 100 on every metric; the raw value is printed above each bar. For latency, lower is better, so a bar below 100 is an improvement.

How to apply it

```
curl http://localhost:11434/api/generate -d '{
  "model": "qwen2.5:0.5b-instruct",
  "prompt": "YOUR PROMPT HERE",
  "stream": false,
  "options": {
    "num_predict": 1024,
    "temperature": 0.0
  }
}'
```

These are request-time settings only. Nothing in your Ollama installation was changed by this benchmark. To use them, pass the options with your API request or set them in your client; no Modelfile edit is required.

9. Alternative configurations

Objective	Configuration	Composite	Observed difference
Maximum Quality	System prompt: Concise-response	6.57	Quality is 8.2% higher than baseline; median latency is 30.6% faster; the quality difference is within run-to-run noise at this sample size.
Maximum Speed	Baseline (model defaults)	4.44	Quality is 0.0% unchanged than baseline; median latency is 0.0% slower; the quality difference is within run-to-run noise at this sample size.
Most Consistent	seed = 42 (fixed)	6.83	Quality is 4.8% higher than baseline; median latency is 31.3% slower; the quality difference is within run-to-run noise at this sample size.

10. Not tested

These techniques were considered but not executed. They are listed so the report cannot be mistaken for a complete sweep of everything Ollama exposes.

Item	Status	Reason
num_gpu (GPU layer offload)	Not tested	Hardware dependent. Choosing a layer count blind would force a model reload and could move the model onto the CPU, invalidating the rest of the benchmark.
keep_alive = 0 (unload between runs)	Not tested	Unloading the model after each request would charge the next configuration with the load time, corrupting the comparison. All runs use keep_alive from settings so the model stays resident.
OLLAMA_NUM_PARALLEL / OLLAMA_MAX_LOADED_MODELS	Not tested	Server-level environment variables. Applying them requires restarting the Ollama service, which this application will not do to your installation.
Concurrent request throughput	Not tested	Benchmark requests are issued with the configured concurrency limit (default 1) so that latency measurements reflect the model, not queueing against itself.
Quantization comparison	Informational / not directly tested	No alternative local quantized variant of this model family is installed, so quantization is reported as information only. This application does not download, convert or replace model files.

11. Representative raw outputs

Every conclusion above rests on text the model actually produced. A sample is reproduced here; the application stores every run in full.

Baseline (model defaults) [baseline]

Options: {"num_predict": 1024}

Run #1 - ok, quality 7.48, latency 3.16 s, 81 tokens, 28.4 tok/s

As an AI language model, I am named Qwen. I was created by Alibaba Cloud based on the knowledge base of artificial intelligence and natural language processing. My purpose is to assist users in generating human-like responses, answering questions, providing information, and performing other tasks that involve language processing and artificial intelligence. If you have any specific questions or need assistance with a particular topic, feel free to ask!

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 3.0; coherence: 9.0

Run #2 - ok, quality 7.48, latency 3.12 s, 81 tokens, 26.6 tok/s

I am Qwen, also known as the "Qwen" model, which translates to "Qwen" in English. As a large language model, I can assist you in many ways, such as answering questions, generating text, performing language translation, playing jokes, helping with creative writing, and many other tasks. I am always ready to assist you with any questions or tasks you might have.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 3.0; coherence: 9.0

Run #3 - ok, quality 8.6, latency 2.49 s, 60 tokens, 25.0 tok/s

I am not a person, but a type of artificial intelligence model created by Alibaba Cloud. I can assist you with questions, provide information, and help with various tasks, such as writing, coding, or even helping you with your daily life. Just let me know what you need help with.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

temperature = 0.0

Options: {"num_predict": 1024, "temperature": 0.0}

Run #1 - ok, quality 8.6, latency 3.61 s, 80 tokens, 22.7 tok/s

I am Qwen, a large language model created by Alibaba Cloud. I am named Qwen, and I am here to assist you with any questions or tasks you may have. I can help you with a wide range of topics, from general knowledge to specific areas such as AI, technology, and more. If you have a question or need help with something specific, feel free to ask!

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

Run #2 - ok, quality 8.6, latency 4.13 s, 80 tokens, 20.0 tok/s

I am Qwen, a large language model created by Alibaba Cloud. I am named Qwen, and I am here to assist you with any questions or tasks you may have. I can help you with a wide range of topics, from general knowledge to specific areas such as AI, technology, and more. If you have a question or need help with something specific, feel free to ask!

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

Run #3 - ok, quality 8.6, latency 3.73 s, 80 tokens, 22.0 tok/s

I am Qwen, a large language model created by Alibaba Cloud. I am named Qwen, and I am here to assist you with any questions or tasks you may have. I can help you with a wide range of topics, from general knowledge to specific areas such as AI, technology, and more. If you have a question or need help with something specific, feel free to ask!

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

seed = 42 (fixed)

Options: {"num_predict": 1024, "seed": 42}

Run #1 - ok, quality 8.6, latency 4.22 s, 82 tokens, 19.8 tok/s

As Qwen, I am a large language model created by Alibaba Cloud based on the knowledge base of the Chinese Natural Language Model Qwen-2. I am designed to help users with a wide range of topics, including but not limited to: AI development, language translation, natural language processing, information retrieval, and much more. If you have any questions or need assistance, feel free to ask me.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

Run #2 - ok, quality 8.6, latency 4.29 s, 82 tokens, 19.6 tok/s

As Qwen, I am a large language model created by Alibaba Cloud based on the knowledge base of the Chinese Natural Language Model Qwen-2. I am designed to help users with a wide range of topics, including but not limited to: AI development, language translation, natural language processing, information retrieval, and much more. If you have any questions or need assistance, feel free to ask me.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

Run #3 - ok, quality 8.6, latency 4.15 s, 82 tokens, 20.3 tok/s

As Qwen, I am a large language model created by Alibaba Cloud based on the knowledge base of the Chinese Natural Language Model Qwen-2. I am designed to help users with a wide range of topics, including but not limited to: AI development, language translation, natural language processing, information retrieval, and much more. If you have any questions or need assistance, feel free to ask me.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

System prompt: Concise-response

Options: {"num_predict": 1024}

Run #1 - ok, quality 8.88, latency 2.75 s, 39 tokens, 21.4 tok/s

I am AI language model. I don't have a name. I am here to assist you with your questions, provide information, and engage in conversation. How can I help you today?

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 10.0; coherence: 9.0

Run #2 - ok, quality 8.88, latency 2.98 s, 56 tokens, 19.5 tok/s

I'm an artificial intelligence, not a person. I don't have a name and I don't exist as a being. I can't help you with names or personal assistance. If you have any questions or need help, please ask in a way that I can understand.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 10.0; coherence: 9.0

Run #3 - ok, quality 8.88, latency 2.19 s, 43 tokens, 20.6 tok/s

I'm a large language model, not a person. I don't have a name, and I cannot help you with personal assistance. If you have a specific question or need assistance, please ask it directly.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 10.0; coherence: 9.0

System prompt: Concise-response + temperature 0.2

Options: {"num_predict": 1024, "temperature": 0.2}

Run #1 - ok, quality 8.6, latency 1.70 s, 33 tokens, 20.7 tok/s

I am an artificial intelligence designed to assist with various tasks and provide information. If you have any questions or need help with something specific, feel free to ask.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

Run #2 - ok, quality 8.6, latency 2.36 s, 43 tokens, 19.1 tok/s

I am an artificial intelligence designed to assist with various tasks and provide information. If you have a specific question or need help with something, please let me know, and I'll do my best to assist you.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 8.6; coherence: 9.0

Run #3 - ok, quality 8.88, latency 2.29 s, 40 tokens, 18.3 tok/s

I am an artificial intelligence designed to assist with various tasks, but I do not have a name. If you have a specific question or need assistance with something, I can help you with that.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 10.0; coherence: 9.0

System prompt: Structured-output

Options: {"num_predict": 1024}

Run #1 - ok, quality 8.88, latency 3.28 s, 50 tokens, 21.0 tok/s

My name is Alexa, and I am an artificial intelligence designed to assist with my users. I can help you by providing information, answering questions, and performing tasks as instructed. If you have any questions or need assistance, feel free to ask!

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 10.0; coherence: 9.0

Run #2 - ok, quality 8.88, latency 2.29 s, 43 tokens, 19.8 tok/s

My name is AI assistant and I can help you with any information you need. If you have a specific question or need help with something, please let me know and I'll do my best to assist you.

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 10.0; coherence: 9.0

Run #3 - ok, quality 8.88, latency 1.62 s, 30 tokens, 19.6 tok/s

My name is AI assistant. I'm here to provide information and assist you with any questions you may have. How can I help you today?

Evaluation - correctness: not scorable; instruction_following: not scorable; completeness: 8.0; format_compliance: 9.0; relevance: 10.0; coherence: 9.0

12. Limitations

Sample size

Each configuration was measured 5 times (top configurations were re-sampled with extra runs). That is enough to expose large differences, but differences of a few percent are likely to be noise. Confidence intervals are reported with every mean; where they overlap, treat the configurations as indistinguishable.

Prompt dependence

Results apply to the prompt that was benchmarked. A configuration that helps a JSON-extraction prompt may hurt creative writing. Re-run the benchmark with a prompt representative of your workload before adopting a setting broadly.

Stochastic generation

Unless a seed is fixed, the model samples differently on every run. Some of the variation between configurations is sampling noise rather than a real effect of the setting.

Evaluator limitations

Quality scores are deterministic heuristics over the visible output: format checks, explicit-constraint checks, answer-pattern matching and repetition detection. They do not verify factual accuracy for open-ended prompts and they do not measure reasoning. A human reading the raw outputs may reasonably disagree with them.

Hardware dependence

Latency and throughput reflect this machine and its load at the time of the run. Results will differ on other hardware, and background activity on this machine can move the numbers.

Runtime and version differences

Measured against Ollama 0.34.2. Other versions may change default parameters, scheduling, and which options are honoured.

Unavailable metrics

The following were not measurable on this system and are reported as N/A rather than estimated: GPU utilisation and GPU memory (nvidia-smi unavailable); time to first token (non-streaming runs).

Scope of changes

Every configuration in this report is a request-time setting. No model file, Modelfile, Ollama setting or system configuration was modified, and nothing needs to be undone.